GA VRPTW Optimizer - Complete Application Suite

A comprehensive graphical and command-line application suite for running and analyzing Genetic Algorithm experiments on Vehicle Routing Problem with Time Windows (VRPTW) benchmark instances.

🏷 What's Included

This package provides three ways to interact with the GA VRPTW solver:

1. **Basic UI Application** (`run_ui.py`)

- Simple, user-friendly graphical interface
- Perfect for beginners and interactive experimentation
- File browser, operator selection, parameter tuning
- Real-time progress tracking and result visualization

2. **Advanced UI Application** (`run_advanced_ui.py`)

- All features from Basic UI plus advanced comparison tools
- Compare multiple instances side-by-side
- Operator performance comparison
- Statistical analysis and visualization

3. **Command-Line Interface** (`ga_cli.py`)

- Perfect for automation and batch processing
- Scriptable for research workflows
- Full parameter control
- Integration with other tools

🚀 Quick Start

PROF

Installation

```
pip install matplotlib numpy
```

Run Basic UI

```
python run_ui.py
```

Run Advanced UI

```
```

```
python run_advanced_ui.py
```

Run from Command Line

```
python ga_cli.py run C101.txt --population 50 --generations 100
python ga_cli.py list instances
python ga_cli.py list operators
```

📁 Directory Structure

```
genetic_algorithm/
├── run_ui.py                          # ← Start here
├── run_advanced_ui.py                 # ← Advanced features
├── ga_cli.py                          # ← Command-line tool
├── check_env.py                       # ← Verify setup
├── examples.py                        # ← Code examples
│
├── QUICK_START.md                     # ← Getting started guide
├── UI_README.md                       # ← Detailed UI guide
├── EXAMPLES.md                        # ← Usage patterns
│
├── data/                              # VRPTW Benchmark instances
│   ├── C101.txt - C109.txt           # Clustered instances
│   ├── R101.txt - R112.txt           # Random instances
│   └── RC101.txt - RC208.txt         # Mixed instances
│
├── results/                           # Experiment results (auto-
created)
│   └── C101/
│       ├── run_001.json
│       ├── run_002.json
│       └── summary.json
│
├── vrptw/                             # VRPTW problem implementation
│   ├── instance.py                   # Problem instance
│   ├── customer.py                   # Customer representation
│   ├── fitness.py                    # Fitness evaluation
│   ├── generateInit.py               # Initialization heuristics
│   └── view/
│       ├── app.py                    # ← Basic UI code
│       └── advanced_app.py           # ← Advanced UI code
│
├── ga/                                # Genetic Algorithm core
│   ├── genetic_algorithm.py          # GA implementation
│   └── selection.py                  # Selection operators
│
└── operators/                         # Genetic operators
```

PROF

```
    ├── crossover.py                # 5 crossover operators
    └── mutation.py                 # 4 mutation operators
```

🎯 Available Operators

Initialization (Population Generation)

Operator	Purpose	When to Use
Random	Greedy random insertion	Quick initial solutions
Solomon	Nearest neighbor insertion	Good quality starting point
Cluster First	Cluster then route	Large-scale problems
Savings	Clarke-Wright savings	Balanced approach
Mixed	Random combination	Diverse population

Crossover (Parent Recombination)

Operator	Type	Best For
PMX	Partially Mapped	TSP-style tours
Order (OX)	Order preserving	Sequential customers
Cycle (CX)	Cycle based	Complex structures
Edge Assembly	Route-based (optimized for VRPTW)	Time window constraints
Route-Based	Route-level operations	Multi-route solutions

Mutation (Local Search)

PROF

Operator	Strategy	Effect
2-Opt	Swap edge pairs	Strong local optimization
Or-Opt	Move segments	Moderate improvement
Insert	Random relocation	Neighbor exploration
Scramble	Shuffle subsequences	Diversification

Selection (Parent Selection)

Method	Mechanism
Tournament	Probabilistic selection via tournaments (default)
Roulette	Fitness-proportionate selection

Method	Mechanism
Truncation	Deterministic best selection

📊 Using the Application

Basic UI Workflow

1. **Select Data Files**
    - Browse the `data/` folder or add custom files
    - Select multiple instances for batch runs
2. **Configure Operators**
    - Choose initialization, crossover, mutation methods
    - Select selection strategy
    - Specify any combination
3. **Set Parameters**
    - Population size: 20-100+
    - Generations: 50-500+
    - Mutation rate: 0.1-0.5
    - Number of runs: 1-10+
4. **Run Experiment**
    - Click "Run Experiment"
    - Monitor progress bar
    - View results automatically
5. **Analyze Results**
    - Switch to Results tab
    - Select instance to inspect
    - View convergence and statistics

PROF

Advanced UI Features

- **Operator Comparison Tab**: Side-by-side performance analysis
- **Multi-instance Comparison**: Compare across benchmarks
- **Statistical Summaries**: Best/worst/average metrics

Command-Line Usage

Single Experiment

```
python ga_cli.py run C101.txt
```

Custom Parameters

```
python ga_cli.py run R101.txt \
    --population 100 \
    --generations 300 \
    --mut-rate 0.15 \
    --runs 10 \
    --init solomon \
    --crossover edge \
    --mutation 2opt
```

Batch Processing

```
#!/bin/bash
for inst in C101 C102 C103 R101 R102; do
    python ga_cli.py run ${inst}.txt --runs 5
done
```

List Options

```
python ga_cli.py list instances    # Show data files
python ga_cli.py list operators    # Show available operators
```

📈 Understanding Results

Saved Files

PROF

Each run produces JSON files in `results/{instance}/`:

- `run_001.json` - Individual run details
- `run_002.json` - Additional runs
- `summary.json` - Aggregated statistics

JSON Structure

```
{
    "timestamp": "2024-04-20T...",
    "instance": "C101.txt",
    "parameters": {...},
    "best_fitness": 828.94,
    "best_solution": [...],
    "best_record": [1000, 950, ..., 828.94],
```

```
    "avg_record": [950, 920, ..., 850],
    "runtime": 15.23
}
```

Visualization

- **Convergence Plot**: Best and average fitness per generation
- **Distribution Plot**: Box plot of final solutions
- **Statistics**: Mean, std dev, min, max

🔧 Customization

Add Custom Operator

```
# In vrptw/view/app.py, add your function to the dictionary:

INITIALIZERS["MyInit"] = my_init_function
CROSSOVERS["MyXover"] = my_crossover_function
MUTATIONS["MyMut"] = my_mutation_function
```

Custom Initialization Example

```
def my_initializer(instance):
    """Your custom initialization logic"""
    solution = []
    # Your implementation
    return solution
```

Programmatic Use

PROF

```
from ga.genetic_algorithm import GeneticAlgorithm
from vrptw.instance import Instance

instance = Instance("data/C101.txt")
ga = GeneticAlgorithm(...)
result = ga.run()
```

⚙ Troubleshooting

UI Won't Start

```
```

```
python check_env.py              # Verify environment
pip install matplotlib numpy     # Install dependencies
python --version                 # Check Python 3.7+
```

Import Errors

```
# Ensure you're in the project root directory
cd /path/to/genetic_algorithm

# Run from correct location
python run_ui.py
```

Missing Data Files

- Ensure `data/` folder exists with `.txt` files
- Use "Add Custom File" for files in other locations

Performance Issues

- Start with smaller population/generations for testing
- Use fewer runs initially
- Test with C101 before C108 (smaller problems first)

📚 Documentation

- **QUICK_START.md** - Step-by-step getting started guide
- **UI_README.md** - Detailed UI documentation
- **examples.py** - Code usage examples
- This file - Complete reference

PROF

🧪 Example Workflows

Quick Test (5 min)

```
python run_ui.py
# Select C101, R101
# Population=20, Generations=50, Runs=1
# View Results after completion
```

Moderate Comparison (30 min)

```
python ga_cli.py run C101.txt --runs 5
python ga_cli.py run C102.txt --runs 5
```

```
python run_ui.py  # View Results tab
```

Comprehensive Study (2 hours+)

```
# Test all combinations
for inst in C101 C102 R101; do
    for init in random solomon; do
        for cross in edge pmx ox; do
            for mut in 2opt oropt; do
                python ga_cli.py run ${inst}.txt \
                    --init $init --crossover $cross \
                    --mutation $mut --runs 5 \
                    --generations 200
            done
        done
    done
done
python run_advanced_ui.py  # Analyze all results
```

🔍 Key Features

✅ **Easy to Use** - Intuitive graphical interface
✅ **Flexible** - 5 initialization, 5 crossover, 4 mutation options
✅ **Powerful** - Multiple GA runs with statistics
✅ **Extensible** - Add custom operators easily
✅ **Scriptable** - Command-line interface for automation
✅ **Comparable** - Built-in operator comparison tools
✅ **Visualizable** - Automatic plotting and analysis
✅ **Documented** - Comprehensive guides and examples

PROF

📞 Support

For issues or questions:

1. Check `QUICK_START.md` for quick answers
2. Review `UI_README.md` for UI-specific help
3. See `examples.py` for code patterns
4. Check `results/` folder for experiment outputs
5. Review error messages and logs carefully

📝 Citation

If you use this application for research, please cite:

```
GA VRPTW Optimizer Suite (2024)
Genetic Algorithm for Vehicle Routing Problem with Time Windows
```

📄 License

As part of the Genetic Algorithm for VRPTW project.

Ready to optimize? Start with: `python run_ui.py`

For command-line: `python ga_cli.py --help`

Questions? See `QUICK_START.md` or `check_env.py`